```
import discord
from discord.ext import commands
from discord import app_commands
import os
import json
from datetime import datetime, timedelta, timezone
from collections import defaultdict

OWNER_ID = 1408144132966322407

def is_owner(user: discord.Member) -> bool:
    return user.id == OWNER_ID

intents = discord.Intents.default()
intents.message_content = True
intents.members = True
intents.bans = True
intents.guilds = True
intents.moderation = True

bot = commands.Bot(command_prefix="!", intents=intents, help_command=None)
tree = bot.tree

WARNINGS_FILE = "warnings.json"
CONFIG_FILE = "config.json"

def load_warnings():
    if os.path.exists(WARNINGS_FILE):
        with open(WARNINGS_FILE, "r") as f:
            return json.load(f)
    return {}

def save_warnings(data):
    with open(WARNINGS_FILE, "w") as f:
        json.dump(data, f, indent=2)

def load_config():
    if os.path.exists(CONFIG_FILE):
        with open(CONFIG_FILE, "r") as f:
            return json.load(f)
    return {"alert_users": []}

def save_config(data):
    with open(CONFIG_FILE, "w") as f:
```

```
        json.dump(data, f, indent=2)

warnings_data = load_warnings()
config_data = load_config()

NUKE_WINDOW = 10
NUKE_THRESHOLD = 3
nuke_tracker = defaultdict(lambda: defaultdict(list))

async def get_audit_executor(guild, action):
    try:
        async for entry in guild.audit_logs(limit=1, action=action):
            if (datetime.now(timezone.utc) - entry.created_at).total_seconds() <
                return entry.user
    except discord.Forbidden:
        pass
    return None

async def check_nuke(guild, user, action_label):
    if user is None or user.bot:
        return
    now = datetime.now(timezone.utc)
    cutoff = now - timedelta(seconds=NUKE_WINDOW)
    tracker = nuke_tracker[guild.id][user.id]
    tracker.append(now)
    nuke_tracker[guild.id][user.id] = [t for t in tracker if t > cutoff]
    count = len(nuke_tracker[guild.id][user.id])
    if count >= NUKE_THRESHOLD:
        nuke_tracker[guild.id][user.id] = []
        log_channel = guild.system_channel
        actions_taken = []
        try:
            member = guild.get_member(user.id)
            if member:
                bot_top_role = guild.me.top_role
                removable = [r for r in member.roles if r.name != "@everyone" and
                if removable:
                    await member.remove_roles(*removable, reason="[Anti-Nuke]")
                    actions_taken.append("Rollen entfernt")
            try:
                await guild.ban(user, reason=f"[Anti-Nuke] {count}x {action_label
                actions_taken.append("Gebannt")
            except discord.Forbidden:
                actions_taken.append("Ban fehlgeschlagen")
            msg = f"Anti-Nuke: {user} - {count}x {action_label} - {', '.join(acti
        except Exception:
            msg = f"Anti-Nuke: {user} - Fehler"
```

```
        if log_channel:
            await log_channel.send(msg)

@bot.event
async def on_member_ban(guild, user):
    executor = await get_audit_executor(guild, discord.AuditLogAction.ban)
    await check_nuke(guild, executor, "Ban")

@bot.event
async def on_guild_channel_delete(channel):
    executor = await get_audit_executor(channel.guild, discord.AuditLogAction.cha
    await check_nuke(channel.guild, executor, "Channel-Loeschung")

@bot.event
async def on_guild_role_delete(role):
    executor = await get_audit_executor(role.guild, discord.AuditLogAction.role_d
    await check_nuke(role.guild, executor, "Rollen-Loeschung")

@bot.event
async def on_member_remove(member):
    executor = await get_audit_executor(member.guild, discord.AuditLogAction.kick
    if executor:
        await check_nuke(member.guild, executor, "Kick")

@bot.event
async def on_ready():
    await tree.sync()
    print(f"Bot ist online als {bot.user}")
    print("Slash Commands synchronisiert!")

@tree.command(name="help", description="Zeigt alle Befehle")
async def help_cmd(interaction: discord.Interaction):
    embed = discord.Embed(title="Bot Befehle", color=discord.Color.blurple())
    embed.add_field(name="Moderation", value="/teamkick /tempmute /unmute /teamwa
    embed.add_field(name="Alerts", value="/setalerts /removealerts", inline=False
    await interaction.response.send_message(embed=embed)

@tree.command(name="hallo", description="Begruessung")
async def hallo(interaction: discord.Interaction):
    await interaction.response.send_message("Hallo!")

@tree.command(name="teamkick", description="Entfernt alle Rollen eines Users")
@app_commands.describe(member="Der User")
async def teamkick(interaction: discord.Interaction, member: discord.Member):
    if not interaction.user.guild_permissions.manage_roles:
        await interaction.response.send_message("Keine Berechtigung!", ephemeral=
        return
```

```
    if is_owner(member):
        await interaction.response.send_message("Der Eigentuemer ist immun!", eph
        return
    roles = [r for r in member.roles if r.name != "@everyone"]
    if not roles:
        await interaction.response.send_message(f"{member.mention} hat keine Roll
        return
    await member.remove_roles(*roles)
    await interaction.response.send_message(f"Alle Rollen von {member.mention} en

@tree.command(name="tempmute", description="Schaltet einen User stumm")
@app_commands.describe(member="Der User", minuten="Dauer in Minuten", grund="Grun
async def tempmute(interaction: discord.Interaction, member: discord.Member, minu
    if not interaction.user.guild_permissions.moderate_members:
        await interaction.response.send_message("Keine Berechtigung!", ephemeral=
        return
    if is_owner(member):
        await interaction.response.send_message("Der Eigentuemer ist immun!", eph
        return
    until = datetime.now(timezone.utc) + timedelta(minutes=minuten)
    try:
        await member.timeout(until, reason=grund)
        await interaction.response.send_message(f"{member.mention} fuer {minuten}
    except discord.Forbidden:
        await interaction.response.send_message("Fehlende Berechtigung.")

@tree.command(name="unmute", description="Hebt den Mute auf")
@app_commands.describe(member="Der User")
async def unmute(interaction: discord.Interaction, member: discord.Member):
    if not interaction.user.guild_permissions.moderate_members:
        await interaction.response.send_message("Keine Berechtigung!", ephemeral=
        return
    if is_owner(member):
        await interaction.response.send_message("Der Eigentuemer ist immun!", eph
        return
    try:
        await member.timeout(None)
        await interaction.response.send_message(f"Mute von {member.mention} aufge
    except discord.Forbidden:
        await interaction.response.send_message("Fehlende Berechtigung.")

@tree.command(name="teamwarn", description="Verwarnt einen User")
@app_commands.describe(member="Der User", grund="Grund")
async def teamwarn(interaction: discord.Interaction, member: discord.Member, grun
    if not interaction.user.guild_permissions.manage_messages:
        await interaction.response.send_message("Keine Berechtigung!", ephemeral=
        return
```

```
    if is_owner(member):
        await interaction.response.send_message("Der Eigentuemer ist immun!", eph
        return
    user_id = str(member.id)
    if user_id not in warnings_data:
        warnings_data[user_id] = []
    warnings_data[user_id].append({"reason": grund, "by": str(interaction.user),
    save_warnings(warnings_data)
    count = len(warnings_data[user_id])
    await interaction.response.send_message(f"{member.mention} verwarnt! Grund: {
    if count >= 3:
        try:
            await member.ban(reason=f"Auto-Ban nach {count} Verwarnungen")
            warnings_data[user_id] = []
            save_warnings(warnings_data)
            await interaction.followup.send(f"{member.mention} automatisch gebann
        except discord.Forbidden:
            await interaction.followup.send("Fehlende Berechtigung zum Bannen.")

@tree.command(name="warnings", description="Zeigt Verwarnungen")
@app_commands.describe(member="Der User")
async def warnings_cmd(interaction: discord.Interaction, member: discord.Member):
    if not interaction.user.guild_permissions.manage_messages:
        await interaction.response.send_message("Keine Berechtigung!", ephemeral=
        return
    user_warnings = warnings_data.get(str(member.id), [])
    if not user_warnings:
        await interaction.response.send_message(f"{member.mention} hat keine Verw
        return
    lines = [f"Verwarnungen fuer {member.mention}: {len(user_warnings)}"]
    for i, w in enumerate(user_warnings, 1):
        lines.append(f"{i}. {w['reason']} - von {w['by']}")
    await interaction.response.send_message("\n".join(lines))

@tree.command(name="allwarnings", description="Alle Verwarnungen")
async def allwarnings(interaction: discord.Interaction):
    if not interaction.user.guild_permissions.manage_messages:
        await interaction.response.send_message("Keine Berechtigung!", ephemeral=
        return
    active = {uid: w for uid, w in warnings_data.items() if w}
    if not active:
        await interaction.response.send_message("Keine aktiven Verwarnungen.")
        return
    embed = discord.Embed(title="Alle Verwarnungen", color=discord.Color.orange()
    for uid, warns in sorted(active.items(), key=lambda x: len(x[1]), reverse=Tru
        try:
            u = await bot.fetch_user(int(uid))
```

```
            name = str(u)
        except Exception:
            name = uid
        embed.add_field(name=f"{name} - {len(warns)}x", value=warns[-1]["reason"]
    await interaction.response.send_message(embed=embed)

@tree.command(name="clearwarnings", description="Loescht Verwarnungen")
@app_commands.describe(member="Der User")
async def clearwarnings(interaction: discord.Interaction, member: discord.Member)
    if not interaction.user.guild_permissions.manage_messages:
        await interaction.response.send_message("Keine Berechtigung!", ephemeral=
        return
    user_id = str(member.id)
    if not warnings_data.get(user_id):
        await interaction.response.send_message(f"{member.mention} hat keine Verw
        return
    count = len(warnings_data[user_id])
    warnings_data[user_id] = []
    save_warnings(warnings_data)
    await interaction.response.send_message(f"{count} Verwarnung(en) von {member.

@tree.command(name="bann", description="Bannt einen User")
@app_commands.describe(member="Der User", grund="Grund")
async def bann(interaction: discord.Interaction, member: discord.Member, grund: s
    if not interaction.user.guild_permissions.ban_members:
        await interaction.response.send_message("Keine Berechtigung!", ephemeral=
        return
    if is_owner(member):
        await interaction.response.send_message("Der Eigentuemer ist immun!", eph
        return
    try:
        await member.ban(reason=grund)
        await interaction.response.send_message(f"{member.mention} gebannt! Grund
    except discord.Forbidden:
        await interaction.response.send_message("Fehlende Berechtigung.")

@tree.command(name="unban", description="Entbannt einen User")
@app_commands.describe(user_id="Die User-ID")
async def unban(interaction: discord.Interaction, user_id: str):
    if not interaction.user.guild_permissions.ban_members:
        await interaction.response.send_message("Keine Berechtigung!", ephemeral=
        return
    try:
        user = await bot.fetch_user(int(user_id))
        await interaction.guild.unban(user)
        await interaction.response.send_message(f"{user} entbannt!")
    except Exception:
```

```
        await interaction.response.send_message("Fehler beim Entbannen.")

@tree.command(name="setalerts", description="DM-Alerts aktivieren")
async def setalerts(interaction: discord.Interaction):
    uid = str(interaction.user.id)
    if uid in config_data["alert_users"]:
        await interaction.response.send_message("Du hast bereits Alerts.")
        return
    config_data["alert_users"].append(uid)
    save_config(config_data)
    await interaction.response.send_message("DM-Alerts aktiviert!")

@tree.command(name="removealerts", description="DM-Alerts deaktivieren")
async def removealerts(interaction: discord.Interaction):
    uid = str(interaction.user.id)
    if uid not in config_data["alert_users"]:
        await interaction.response.send_message("Keine aktiven Alerts.")
        return
    config_data["alert_users"].remove(uid)
    save_config(config_data)
    await interaction.response.send_message("DM-Alerts deaktiviert.")

token = os.environ.get("DISCORD_TOKEN")
if not token:
    raise RuntimeError("DISCORD_TOKEN nicht gesetzt!")

bot.run(token)
```